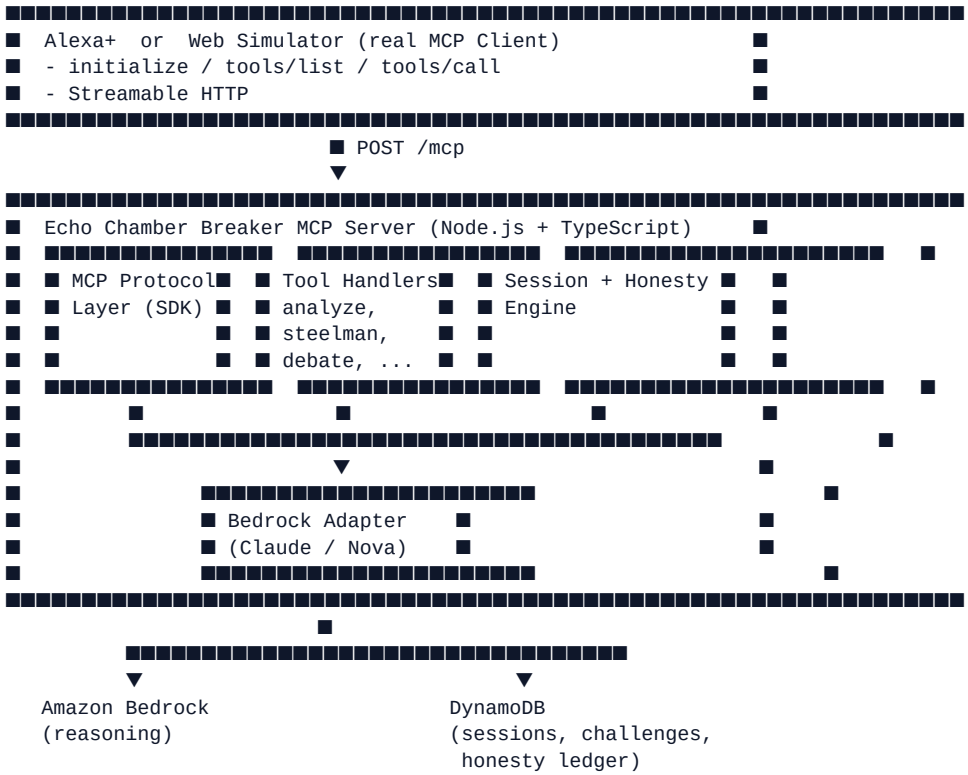
ECHO CHAMBER BREAKER

Technical Architecture • Feasibility • Scalability

Designed for execution with Google Antigravity (agent-first)

Focus	MCP Streamable HTTP server + Bedrock + Simulator
Build Style	Antigravity-first (phase prompts + artifact review)
Target	Submission-ready in 8–12 focused days
Companion	Product Definition PDF + Pitch Deck

1. System Architecture Overview



The simulator is a first-class citizen: it must speak real MCP, not fake results. This is both a product requirement and a judging requirement.

2. Component Breakdown

Component	Responsibility	Tech
MCP Protocol Layer	Streamable HTTP transport, initialize, tools/list, tools/call, session lifecycle	@modelcontextprotocol/sdk
Tool Handlers	Business logic for each of the 9 tools; return say + structuredContent	TypeScript modules
Bedrock Adapter	All LLM calls; prompt templates; model switching; local mock mode	AWS SDK / Strands
Honesty Engine	Score calculation, ledger append, report generation	Pure TS + store
Session Store	Challenges, debate sessions, user profile	DynamoDB or JSON/SQLite
Web Simulator	Real MCP client UI + live trace + score visualization	Vite + React or HTML/JS

Critical Design Rules

- Every tool response includes a short voice-ready 'say' string.
- Bedrock is behind an adapter so the system can run without AWS keys (mock mode) for fast local iteration.
- Honesty events are append-only. Current score is derived.
- MCP requests are largely stateless at the protocol level; application state lives in the store keyed by session/challenge IDs.
- The simulator must not invent tool results.

3. Feasibility Analysis

Why this is feasible in a hackathon window

- MCP Streamable HTTP is well-supported by the official TypeScript SDK with working examples.
- Core value can be demonstrated with only 2–3 tools (analyze_stance + steelman + honesty_report).
- Bedrock has clear SDKs and the AWS Builder mini-challenge rewards documented usage.
- A polished simulator is pure frontend work that can be developed in parallel with the server.
- Local JSON/SQLite store removes AWS dependency for the first several days.

Highest-Risk Areas (and mitigations)

Risk	Mitigation
Incorrect Streamable HTTP implementation	Start from official SDK example; verify with MCP Inspector early
Weak steelman quality	Invest time in system prompts; test with real opinions; keep a prompt lab
Bedrock latency	Use faster model for analyze_stance; show status in UI; cache recent results
Scope creep on multi-turn debate	Ship single-turn steelman first; debate is phase 2 enhancement
Simulator looks fake	Force real MCP calls + prominent live trace panel

4. Scalability Considerations

For the hackathon the system only needs to support a small number of concurrent demo users. The architecture is still chosen so it can grow.

Current (Hackathon) Scale

- Single Node process or simple serverless function is enough.
- DynamoDB on-demand or local file store is fine.
- Bedrock calls are the main cost and latency driver — keep them intentional.

Growth Path

Dimension	Hackathon	Later
Compute	Single Node / small container	Horizontal pods or Lambda + API Gateway
State	DynamoDB or local JSON	DynamoDB + optional Redis for hot sessions
LLM	Direct Bedrock calls	Caching, model routing, async jobs for long debates
Auth	Optional API key / none for demo	OAuth / Alexa+ account linking path
Observability	Basic logs + friction log	Structured traces, latency histograms, score analytics

Do not over-engineer for scale during the hackathon. Correct protocol + strong steelman + clear demo beat premature distributed systems.

5. Building with Antigravity (Agent-First)

Antigravity is well-suited to this project because the work is phaseable, the interfaces are clear (MCP tools + store + Bedrock adapter), and verification can be done with the MCP Inspector and a browser agent.

Recommended Antigravity Workflow

- Create a project and paste the Product Definition + this Architecture document as context.
- Execute one phase at a time. Never ask for the entire system in a single prompt.
- After each phase, require artifacts: file tree, key source files, and a short verification checklist.
- Use the browser / terminal capabilities to run the MCP server and hit it with the Inspector or curl.
- Keep a running FRICTION_LOG.md that the agent updates whenever something is harder than expected.

Phase Prompt Strategy

Give Antigravity a narrow, testable goal plus acceptance criteria. Example for Phase 1:

```
Goal: Implement a TypeScript MCP server using the official SDK with Streamable HTTP.
Acceptance:
- initialize returns correct protocol version and serverInfo
- tools/list returns analyze_stance and steelman
- tools/call for steelman returns both a short 'say' field and structured content
- Local mock mode works without AWS credentials
- README contains exact run commands
Produce: full source for the server + a short verification script.
```

What to Watch When Using Agents

- MCP transport details (headers, Accept, JSON-RPC shape) — verify early.
- Prompt quality for steelmanning — agents often produce generic counters; force iterative improvement.
- State management across calls — ensure session/challenge IDs are actually persisted and reused.
- Simulator authenticity — reject any frontend that invents tool results.

6. Recommended Stack & Repository Shape

```
echo-chamber-breaker/
■■■ README.md
■■■ LICENSE (MIT)
■■■ package.json / tsconfig.json
■■■ .env.example
■■■ src/
■   ■■■ index.ts                # entry + HTTP
■   ■■■ mcp/server.ts           # McpServer setup
■   ■■■ mcp/tools/              # one module per tool group
■   ■■■ services/bedrock.ts
■   ■■■ services/honestyEngine.ts
■   ■■■ services/sessionStore.ts
■   ■■■ types/
■■■ simulator/                  # real MCP client UI
■■■ scripts/demo.sh
■■■ docs/
■   ■■■ ARCHITECTURE.md
■   ■■■ FRICTION_LOG.md
■   ■■■ PRODUCT_FEEDBACK.md
■■■ tests/
```

Stack Summary

Layer	Choice
Language	TypeScript on Node 20+
MCP	@modelcontextprotocol/sdk (Streamable HTTP, 2025-11-25+)
LLM	Amazon Bedrock (Claude / Nova) + local mock
State	DynamoDB (prod) / JSON or SQLite (dev)
Simulator	Vite + React or lightweight HTML/JS
Agent build	Google Antigravity (primary)

7. Verification & Done Criteria

Minimum Viable Demo (must all be true)

- MCP server starts with one command and responds to initialize + tools/list.
- At least analyze_stance and steelman work end-to-end (real or mocked Bedrock).
- Simulator connects as a real client and shows a live tool-call trace.
- Honesty score can be updated and reported.
- README lets a stranger reproduce the demo in a few minutes.

Strong Submission Bar

- All planned tools implemented with consistent say + structuredContent.
- Documented Bedrock usage (AWS Builder).
- Friction log with real problems and solutions.
- Polished dark-theme simulator with score visualization.
- Clear 2–3 minute demo video showing the protocol in action.

Architecture is intentionally simple so Antigravity (and a human) can execute it quickly and correctly. Depth lives in the steelman quality, the honesty loop, and the authenticity of the MCP demo — not in distributed complexity.

Echo Chamber Breaker — Architecture for agent-first execution.